Mini-projet: Classification de feuilles d'arbres

Ce notebook est concu pour:

- entrainer plusieurs architectures CNN,
- varier les couches et les hyperparametres,
- comparer les performances,
- produire des elements exploitables dans le compte-rendu.

> Objectif vise: precision de validation/test > 75%.

In [1]:
```
import os
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from pathlib import Path
from datetime import datetime

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

from sklearn.metrics import classification_report, confusion_matrix

print('TensorFlow version:', tf.__version__)
```

```
TensorFlow version: 2.21.0
```

In [2]:
```
# ----------------------------
# Configuration generale
# ----------------------------
SEED = 42
IMG_SIZE = (128, 128)
BATCH_SIZE = 32
AUTOTUNE = tf.data.AUTOTUNE

# IMPORTANT:
# DATA_DIR doit pointer vers un dossier avec structure:
# DATA_DIR/
#   classe_1/*.jpg
#   classe_2/*.jpg
#   ...
# Chaque sous-dossier = une classe

DATA_DIR = "./data_32_classes"  # ex: r"C:\\Users\\ha054\\Documents\\fdarbre\\
```

```
# Dossiers candidats automatiques (au cas ou tu as deja place tes donnees)
CANDIDATE_DIRS = [
    r"C:\Users\ha054\Documents\fdarbre\leaves_4classes",
    r"C:\Users\ha054\Documents\fdarbre\leaves_32classes",
    r"C:\Users\ha054\Documents\fdarbre\dataset",
    r"C:\Users\ha054\Documents\fdarbre\data",
]

np.random.seed(SEED)
tf.random.set_seed(SEED)

for c in CANDIDATE_DIRS:
    if DATA_DIR is None and Path(c).exists():
        DATA_DIR = c

print('DATA_DIR actuel =', DATA_DIR)
if DATA_DIR is None:
    print('-> Definis DATA_DIR manuellement avant de lancer les cellules suiva
```

```
DATA_DIR actuel = ./data_32_classes
```

In [3]:
```
# ----------------------------
# Chargement des donnees
# ----------------------------
def make_datasets(data_dir, img_size=IMG_SIZE, batch_size=BATCH_SIZE, seed=SEE
    train_ds = tf.keras.utils.image_dataset_from_directory(
        data_dir,
        validation_split=0.2,
        subset='training',
        seed=seed,
        image_size=img_size,
        batch_size=batch_size,
        label_mode='int',
    )

    val_ds = tf.keras.utils.image_dataset_from_directory(
        data_dir,
        validation_split=0.2,
        subset='validation',
        seed=seed,
        image_size=img_size,
        batch_size=batch_size,
        label_mode='int',
    )

    class_names = train_ds.class_names

    # Sous-division validation -> validation + test (50/50)
    val_batches = tf.data.experimental.cardinality(val_ds).numpy()
    test_batches = max(1, val_batches // 2)

    test_ds = val_ds.take(test_batches)
    val_ds_final = val_ds.skip(test_batches)
```

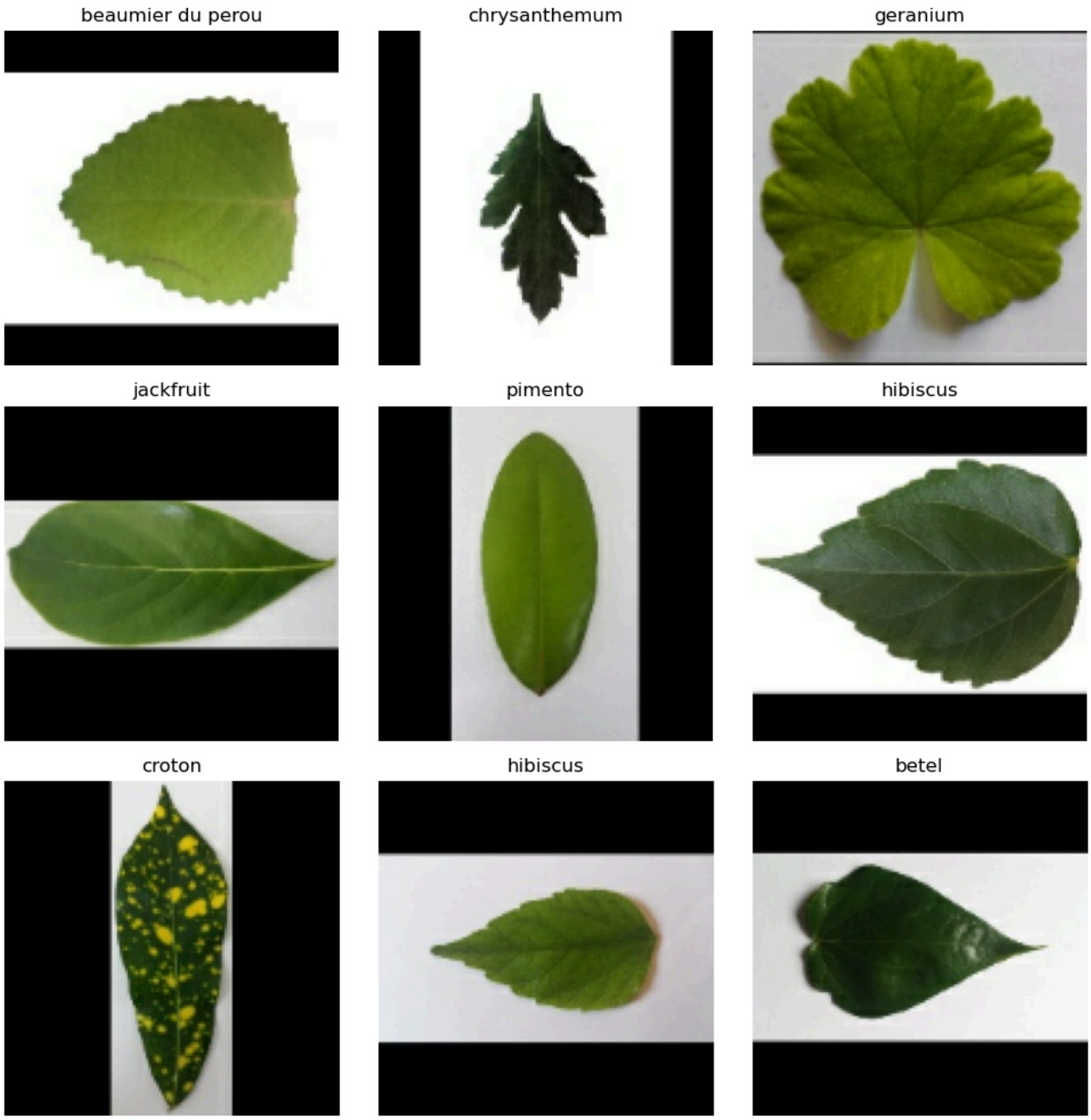
```
    train_ds = train_ds.cache().shuffle(1000, seed=seed).prefetch(buffer_size=
    val_ds_final = val_ds_final.cache().prefetch(buffer_size=AUTOTUNE)
    test_ds = test_ds.cache().prefetch(buffer_size=AUTOTUNE)

    return train_ds, val_ds_final, test_ds, class_names

if DATA_DIR is not None:
    train_ds, val_ds, test_ds, class_names = make_datasets(DATA_DIR)
    num_classes = len(class_names)
    print('Classes:', class_names)
    print('Nombre de classes:', num_classes)
else:
    print('DATA_DIR non defini.')
```

```
Found 4955 files belonging to 32 classes.
Using 3964 files for training.
Found 4955 files belonging to 32 classes.
Using 991 files for validation.
Classes: ['ashanti blood', 'barbados cherry', 'beaumier du perou', 'betel', 'bi
tter orange', 'caricature plant', 'chinese guava', 'chocolate tree', 'chrysanth
emum', 'coeur demoiselle', 'coffee', 'croton', 'duranta gold', 'eggplant', 'fic
us', 'fruitcitere', 'geranium', 'guava', 'hibiscus', 'jackfruit', 'ketembilla',
'lychee', 'mulberry leaf', 'papaya', 'pimento', 'pomme jacquot', 'rose', 'star
apple', 'sweet olive', 'sweet potato', 'thevetia', 'vieux garcon']
Nombre de classes: 32
```

In [4]:
```
# Visualisation rapide d'un batch
if DATA_DIR is not None:
    plt.figure(figsize=(10, 10))
    for images, labels in train_ds.take(1):
        for i in range(min(9, len(images))):
            ax = plt.subplot(3, 3, i + 1)
            plt.imshow(images[i].numpy().astype('uint8'))
            plt.title(class_names[labels[i]])
            plt.axis('off')
    plt.tight_layout()
    plt.show()
```



In [5]:
```
# ----------------------------
# Architectures (variation couches)
# ----------------------------
def build_model(variant='A', num_classes=4, input_shape=(128, 128, 3), dropout
    data_aug = keras.Sequential([
        layers.RandomFlip('horizontal'),
        layers.RandomRotation(0.08),
        layers.RandomZoom(0.1),
    ], name='augmentation')

    inputs = keras.Input(shape=input_shape)
    x = data_aug(inputs)
    x = layers.Rescaling(1.0 / 255)(x)

    if variant == 'A':
```

```
        # 2 blocs Conv
        x = layers.Conv2D(32, 3, activation='relu', padding='same')(x)
        x = layers.MaxPooling2D()(x)
        x = layers.Conv2D(64, 3, activation='relu', padding='same')(x)
        x = layers.MaxPooling2D()(x)

    elif variant == 'B':
        # 3 blocs Conv
        x = layers.Conv2D(32, 3, activation='relu', padding='same')(x)
        x = layers.MaxPooling2D()(x)
        x = layers.Conv2D(64, 3, activation='relu', padding='same')(x)
        x = layers.MaxPooling2D()(x)
        x = layers.Conv2D(128, 3, activation='relu', padding='same')(x)
        x = layers.MaxPooling2D()(x)

    elif variant == 'C':
        # 3 blocs Conv + Dropout
        x = layers.Conv2D(32, 3, activation='relu', padding='same')(x)
        x = layers.MaxPooling2D()(x)
        x = layers.Conv2D(64, 3, activation='relu', padding='same')(x)
        x = layers.MaxPooling2D()(x)
        x = layers.Conv2D(128, 3, activation='relu', padding='same')(x)
        x = layers.MaxPooling2D()(x)
        x = layers.Dropout(dropout)(x)

    else:
        raise ValueError('variant doit etre A, B ou C')

    x = layers.Flatten()(x)
    x = layers.Dense(128, activation='relu')(x)
    if variant == 'C':
        x = layers.Dropout(dropout)(x)
    outputs = layers.Dense(num_classes, activation='softmax')(x)

    return keras.Model(inputs, outputs, name=f'model_{variant}')
```

In [6]:

```
# ----------------------------
# Entrainement + comparaison (variation parametres)
# ----------------------------
EXPERIMENTS = [
    # Étude du nombre d'epochs avec une architecture simple
    {'name': 'P1_ep3_A_adam',  'variant': 'A', 'optimizer': 'adam', 'lr': 1e-3
    {'name': 'P1_ep5_A_adam',  'variant': 'A', 'optimizer': 'adam', 'lr': 1e-3
    {'name': 'P1_ep10_A_adam', 'variant': 'A', 'optimizer': 'adam', 'lr': 1e-3

    # Étude de la profondeur du réseau
    {'name': 'P1_2blocs_A_adam', 'variant': 'A', 'optimizer': 'adam', 'lr': 1e
    {'name': 'P1_3blocs_B_adam', 'variant': 'B', 'optimizer': 'adam', 'lr': 1e

    # Étude du dropout
    {'name': 'P1_3blocs_C_dropout03', 'variant': 'C', 'optimizer': 'adam', 'lr

    # Étude de l'optimiseur
```

```
    {'name': 'P1_A_rmsprop', 'variant': 'A', 'optimizer': 'rmsprop', 'lr': 1e-
]


def get_optimizer(name, lr):
    if name.lower() == 'adam':
        return keras.optimizers.Adam(learning_rate=lr)
    if name.lower() == 'rmsprop':
        return keras.optimizers.RMSprop(learning_rate=lr)
    raise ValueError(f'Optimiseur inconnu: {name}')


def train_one_experiment(exp, data_dir):
    train_ds_local, val_ds_local, test_ds_local, class_names_local = make_data
        data_dir,
        img_size=IMG_SIZE,
        batch_size=exp['batch_size'],
        seed=SEED,
    )

    model = build_model(
        variant=exp['variant'],
        num_classes=len(class_names_local),
        input_shape=(*IMG_SIZE, 3),
        dropout=exp['dropout'],
    )

    optimizer = get_optimizer(exp['optimizer'], exp['lr'])
    model.compile(
        optimizer=optimizer,
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy'],
    )

    callbacks = [
        keras.callbacks.EarlyStopping(monitor='val_accuracy', patience=5, rest
    ]

    history = model.fit(
        train_ds_local,
        validation_data=val_ds_local,
        epochs=exp['epochs'],
        callbacks=callbacks,
        verbose=1,
    )

    test_loss, test_acc = model.evaluate(test_ds_local, verbose=0)

    best_val_acc = float(np.max(history.history['val_accuracy']))
    best_train_acc = float(np.max(history.history['accuracy']))

    result = {
        'name': exp['name'],
```

```
        'variant': exp['variant'],
        'optimizer': exp['optimizer'],
        'lr': exp['lr'],
        'batch_size': exp['batch_size'],
        'epochs_target': exp['epochs'],
        'epochs_ran': len(history.history['loss']),
        'best_train_acc': best_train_acc,
        'best_val_acc': best_val_acc,
        'test_acc': float(test_acc),
        'test_loss': float(test_loss),
    }

    return model, history, result, class_names_local, test_ds_local


all_results = []
all_histories = {}
all_models = {}

if DATA_DIR is not None:
    for exp in EXPERIMENTS:
        print('\n' + '=' * 80)
        print('Lancement:', exp['name'])
        model, history, result, class_names, test_ds = train_one_experiment(ex
        all_results.append(result)
        all_histories[exp['name']] = history.history
        all_models[exp['name']] = model

    results_df = pd.DataFrame(all_results).sort_values(by='best_val_acc', asce
    display(results_df)

    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
    csv_name = f'resultats_cnn_{timestamp}.csv'
    json_name = f'resultats_cnn_{timestamp}.json'
    hist_name = f'historiques_cnn_{timestamp}.json'

    results_df.to_csv(csv_name, index=False)
    with open(json_name, 'w', encoding='utf-8') as f:
        json.dump(all_results, f, indent=2)
    with open(hist_name, 'w', encoding='utf-8') as f:
        json.dump(all_histories, f, indent=2)

    print(f'Fichiers sauves: {csv_name}, {json_name}, {hist_name}')
else:
    print('Definis DATA_DIR avant execution.')
```

```
================================================================================
=
Lancement: P1_ep3_A_adam
Found 4955 files belonging to 32 classes.
Using 3964 files for training.
Found 4955 files belonging to 32 classes.
Using 991 files for validation.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows fo
r TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. P
lease use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/3
124/124 ━━━━━━━━━━━━━━━━━━━━ 15s 106ms/step - accuracy: 0.1393 - loss: 3.1818 -
val_accuracy: 0.3620 - val_loss: 2.2746
Epoch 2/3
124/124 ━━━━━━━━━━━━━━━━━━━━ 12s 100ms/step - accuracy: 0.4281 - loss: 1.9139 -
val_accuracy: 0.4521 - val_loss: 1.6513
Epoch 3/3
124/124 ━━━━━━━━━━━━━━━━━━━━ 12s 95ms/step - accuracy: 0.5802 - loss: 1.3468 -
val_accuracy: 0.6223 - val_loss: 1.1453

================================================================================
=
Lancement: P1_ep5_A_adam
Found 4955 files belonging to 32 classes.
Using 3964 files for training.
Found 4955 files belonging to 32 classes.
Using 991 files for validation.
Epoch 1/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 15s 102ms/step - accuracy: 0.1097 - loss: 3.2926 -
val_accuracy: 0.2211 - val_loss: 2.6752
Epoch 2/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 13s 107ms/step - accuracy: 0.3623 - loss: 2.1443 -
val_accuracy: 0.4814 - val_loss: 1.7014
Epoch 3/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 13s 104ms/step - accuracy: 0.5277 - loss: 1.4892 -
val_accuracy: 0.5656 - val_loss: 1.3632
Epoch 4/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 13s 102ms/step - accuracy: 0.6476 - loss: 1.1414 -
val_accuracy: 0.5910 - val_loss: 1.2434
Epoch 5/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 13s 102ms/step - accuracy: 0.7124 - loss: 0.9234 -
val_accuracy: 0.6928 - val_loss: 0.9903

================================================================================
=
Lancement: P1_ep10_A_adam
Found 4955 files belonging to 32 classes.
Using 3964 files for training.
Found 4955 files belonging to 32 classes.
Using 991 files for validation.
Epoch 1/10
124/124 ━━━━━━━━━━━━━━━━━━━━ 15s 101ms/step - accuracy: 0.1773 - loss: 2.9889 -
val_accuracy: 0.3640 - val_loss: 2.1842
Epoch 2/10
```

124/124 ━━━━━━━━━━━━━━━━━━━━ **12s** 96ms/step - accuracy: 0.4306 - loss: 1.9007 - val_accuracy: 0.5714 - val_loss: 1.5002
Epoch 3/10
124/124 ━━━━━━━━━━━━━━━━━━━━ **12s** 99ms/step - accuracy: 0.5686 - loss: 1.3922 - val_accuracy: 0.6282 - val_loss: 1.2822
Epoch 4/10
124/124 ━━━━━━━━━━━━━━━━━━━━ **12s** 97ms/step - accuracy: 0.6582 - loss: 1.0971 - val_accuracy: 0.6575 - val_loss: 1.1150
Epoch 5/10
124/124 ━━━━━━━━━━━━━━━━━━━━ **12s** 97ms/step - accuracy: 0.7278 - loss: 0.8914 - val_accuracy: 0.7006 - val_loss: 0.9802
Epoch 6/10
124/124 ━━━━━━━━━━━━━━━━━━━━ **12s** 99ms/step - accuracy: 0.7666 - loss: 0.7238 - val_accuracy: 0.7769 - val_loss: 0.7521
Epoch 7/10
124/124 ━━━━━━━━━━━━━━━━━━━━ **13s** 104ms/step - accuracy: 0.8035 - loss: 0.6190 - val_accuracy: 0.8023 - val_loss: 0.6202
Epoch 8/10
124/124 ━━━━━━━━━━━━━━━━━━━━ **12s** 97ms/step - accuracy: 0.8272 - loss: 0.5367 - val_accuracy: 0.8160 - val_loss: 0.5620
Epoch 9/10
124/124 ━━━━━━━━━━━━━━━━━━━━ **13s** 108ms/step - accuracy: 0.8451 - loss: 0.4672 - val_accuracy: 0.8415 - val_loss: 0.5169
Epoch 10/10
124/124 ━━━━━━━━━━━━━━━━━━━━ **14s** 110ms/step - accuracy: 0.8557 - loss: 0.4336 - val_accuracy: 0.8337 - val_loss: 0.5208

==========
=
Lancement: P1_2blocs_A_adam
Found 4955 files belonging to 32 classes.
Using 3964 files for training.
Found 4955 files belonging to 32 classes.
Using 991 files for validation.
Epoch 1/5
124/124 ━━━━━━━━━━━━━━━━━━━━ **15s** 105ms/step - accuracy: 0.2023 - loss: 2.8723 - val_accuracy: 0.3679 - val_loss: 2.1164
Epoch 2/5
124/124 ━━━━━━━━━━━━━━━━━━━━ **13s** 101ms/step - accuracy: 0.4425 - loss: 1.8426 - val_accuracy: 0.5421 - val_loss: 1.5229
Epoch 3/5
124/124 ━━━━━━━━━━━━━━━━━━━━ **13s** 102ms/step - accuracy: 0.5790 - loss: 1.3483 - val_accuracy: 0.6477 - val_loss: 1.1477
Epoch 4/5
124/124 ━━━━━━━━━━━━━━━━━━━━ **13s** 105ms/step - accuracy: 0.6733 - loss: 1.0420 - val_accuracy: 0.6869 - val_loss: 0.9904
Epoch 5/5
124/124 ━━━━━━━━━━━━━━━━━━━━ **12s** 99ms/step - accuracy: 0.7596 - loss: 0.7773 - val_accuracy: 0.7202 - val_loss: 0.8352

==========
=
Lancement: P1_3blocs_B_adam
Found 4955 files belonging to 32 classes.

```
Using 3964 files for training.
Found 4955 files belonging to 32 classes.
Using 991 files for validation.
Epoch 1/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 14s 94ms/step - accuracy: 0.1726 - loss: 2.9121 -
val_accuracy: 0.3464 - val_loss: 2.2079
Epoch 2/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 11s 91ms/step - accuracy: 0.4541 - loss: 1.8195 -
val_accuracy: 0.5577 - val_loss: 1.4876
Epoch 3/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 11s 91ms/step - accuracy: 0.5838 - loss: 1.3303 -
val_accuracy: 0.6243 - val_loss: 1.2672
Epoch 4/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 11s 90ms/step - accuracy: 0.6768 - loss: 1.0193 -
val_accuracy: 0.6673 - val_loss: 1.0558
Epoch 5/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 11s 91ms/step - accuracy: 0.7278 - loss: 0.8312 -
val_accuracy: 0.6732 - val_loss: 1.0230

================================================================================
=
Lancement: P1_3blocs_C_dropout03
Found 4955 files belonging to 32 classes.
Using 3964 files for training.
Found 4955 files belonging to 32 classes.
Using 991 files for validation.
Epoch 1/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 14s 95ms/step - accuracy: 0.1549 - loss: 3.0419 -
val_accuracy: 0.3249 - val_loss: 2.2691
Epoch 2/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 12s 98ms/step - accuracy: 0.3645 - loss: 2.1462 -
val_accuracy: 0.5225 - val_loss: 1.7507
Epoch 3/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 12s 96ms/step - accuracy: 0.5013 - loss: 1.5988 -
val_accuracy: 0.6732 - val_loss: 1.1093
Epoch 4/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 12s 94ms/step - accuracy: 0.5928 - loss: 1.2565 -
val_accuracy: 0.7045 - val_loss: 0.9671
Epoch 5/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 12s 98ms/step - accuracy: 0.6650 - loss: 1.0613 -
val_accuracy: 0.8004 - val_loss: 0.7325

================================================================================
=
Lancement: P1_A_rmsprop
Found 4955 files belonging to 32 classes.
Using 3964 files for training.
Found 4955 files belonging to 32 classes.
Using 991 files for validation.
Epoch 1/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 13s 92ms/step - accuracy: 0.1726 - loss: 3.1566 -
val_accuracy: 0.3542 - val_loss: 2.3721
Epoch 2/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 12s 96ms/step - accuracy: 0.3948 - loss: 2.0513 -
```

```
val_accuracy: 0.4736 - val_loss: 1.8095
Epoch 3/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 11s 88ms/step - accuracy: 0.5199 - loss: 1.5742 -
val_accuracy: 0.6360 - val_loss: 1.2781
Epoch 4/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 11s 89ms/step - accuracy: 0.6075 - loss: 1.3198 -
val_accuracy: 0.6614 - val_loss: 1.0926
Epoch 5/5
124/124 ━━━━━━━━━━━━━━━━━━━━ 11s 89ms/step - accuracy: 0.6650 - loss: 1.0635 -
val_accuracy: 0.7045 - val_loss: 0.9059
```

	name	variant	optimizer	lr	batch_size	epochs_target	epo
2	P1_ep10_A_adam	A	adam	0.001	32	10	
5	P1_3blocs_C_dropout03	C	adam	0.001	32	5	
3	P1_2blocs_A_adam	A	adam	0.001	32	5	
6	P1_A_rmsprop	A	rmsprop	0.001	32	5	
1	P1_ep5_A_adam	A	adam	0.001	32	5	
4	P1_3blocs_B_adam	B	adam	0.001	32	5	
0	P1_ep3_A_adam	A	adam	0.001	32	3	

```
Fichiers sauves: resultats_cnn_20260507_234315.csv, resultats_cnn_20260507_2343
15.json, historiques_cnn_20260507_234315.json
```

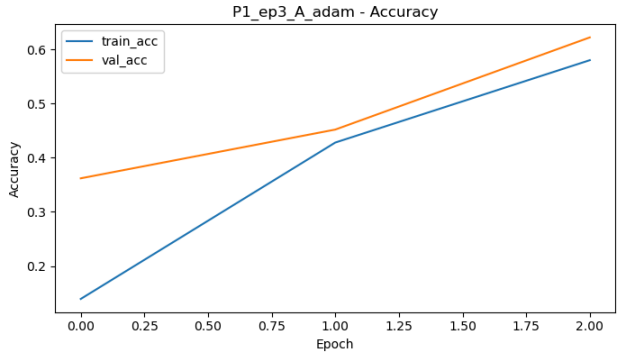
In [7]:

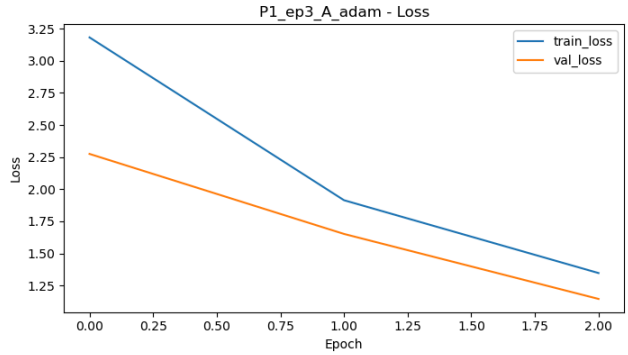
```python
# Courbes d'apprentissage des experiences
if DATA_DIR is not None and len(all_histories) > 0:
    n = len(all_histories)
    plt.figure(figsize=(14, 4 * n))

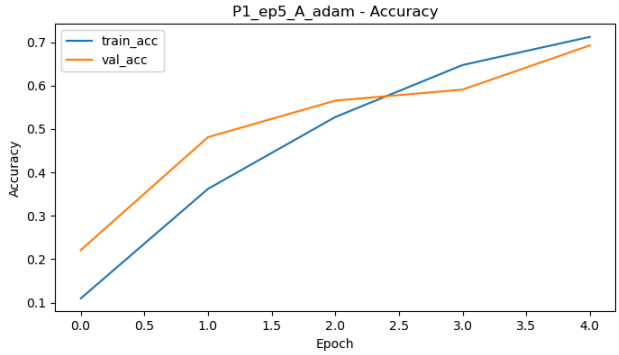
    for i, (name, hist) in enumerate(all_histories.items(), start=1):
        plt.subplot(n, 2, 2 * i - 1)
        plt.plot(hist['accuracy'], label='train_acc')
        plt.plot(hist['val_accuracy'], label='val_acc')
        plt.title(f'{name} - Accuracy')
        plt.xlabel('Epoch')
        plt.ylabel('Accuracy')
        plt.legend()

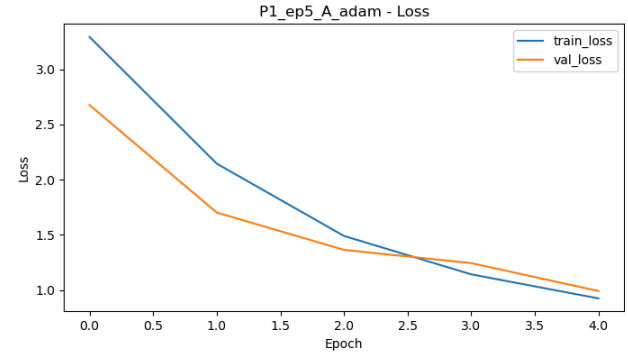
        plt.subplot(n, 2, 2 * i)
        plt.plot(hist['loss'], label='train_loss')
        plt.plot(hist['val_loss'], label='val_loss')
        plt.title(f'{name} - Loss')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.legend()

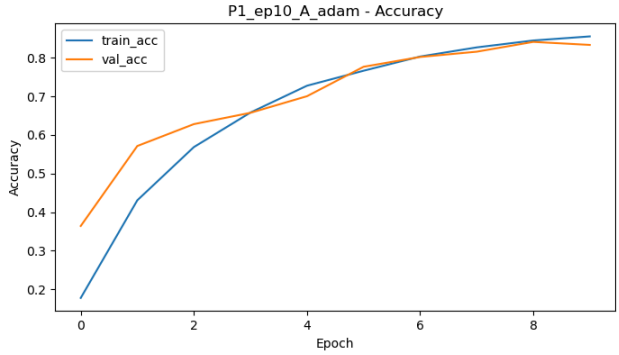
    plt.tight_layout()
    plt.show()
```

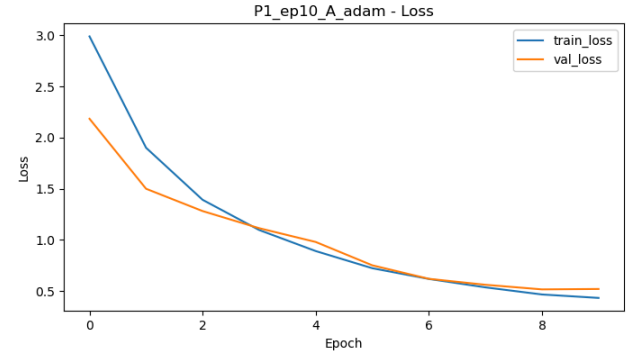

P1_ep3_A_adam - Accuracy
train_acc
val_acc
Accuracy
Epoch

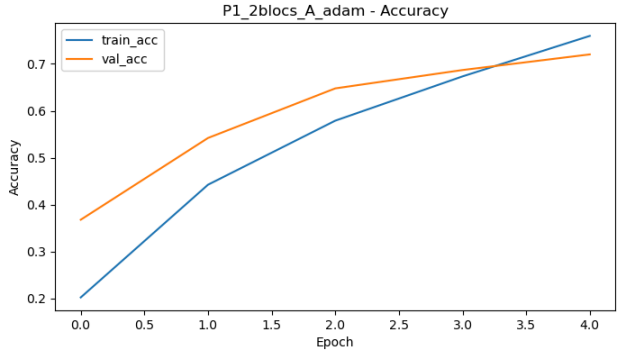

P1_ep3_A_adam - Loss
train_loss
val_loss
Loss
Epoch

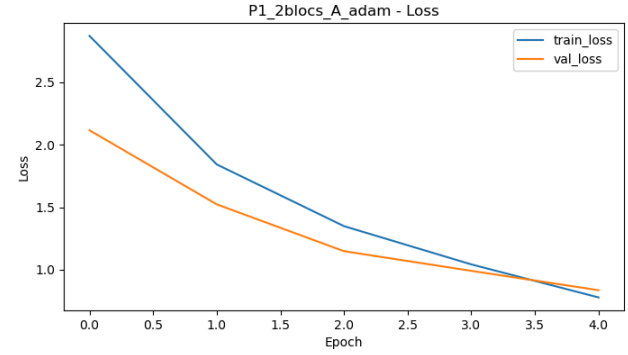

P1_ep5_A_adam - Accuracy
train_acc
val_acc
Accuracy
Epoch

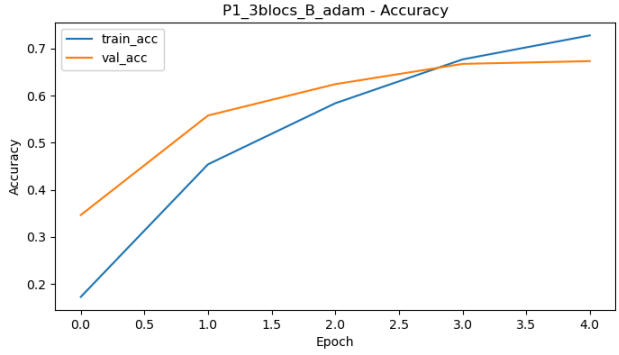

P1_ep5_A_adam - Loss
train_loss
val_loss
Loss
Epoch

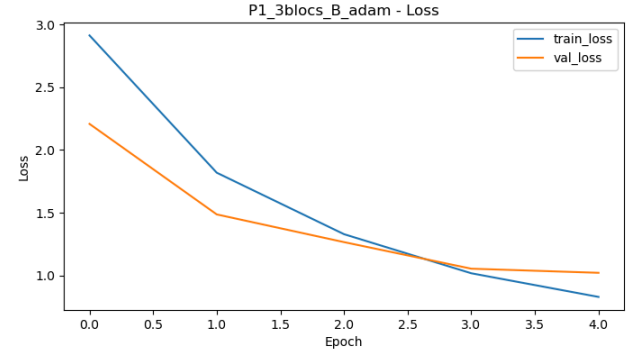

P1_ep10_A_adam - Accuracy
train_acc
val_acc
Accuracy
Epoch

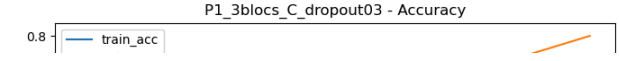

P1_ep10_A_adam - Loss
train_loss
val_loss
Loss
Epoch

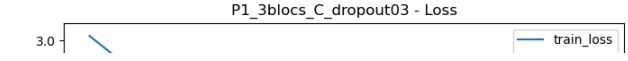

P1_2blocs_A_adam - Accuracy
train_acc
val_acc
Accuracy
Epoch


P1_2blocs_A_adam - Loss
train_loss
val_loss
Loss
Epoch


P1_3blocs_B_adam - Accuracy
train_acc
val_acc
Accuracy
Epoch


P1_3blocs_B_adam - Loss
train_loss
val_loss
Loss
Epoch


P1_3blocs_C_dropout03 - Accuracy
train_acc


P1_3blocs_C_dropout03 - Loss
train_loss

In [8]:
```
# Matrice de confusion + classification report pour le meilleur modele
if DATA_DIR is not None and len(all_results) > 0:
    best_exp_name = pd.DataFrame(all_results).sort_values('best_val_acc', asce
    best_model = all_models[best_exp_name]

    # Reconstruit un test_ds avec le meme batch_size que l'experience gagnante
    best_exp = next(x for x in EXPERIMENTS if x['name'] == best_exp_name)
    _, _, best_test_ds, best_class_names = make_datasets(DATA_DIR, batch_size=

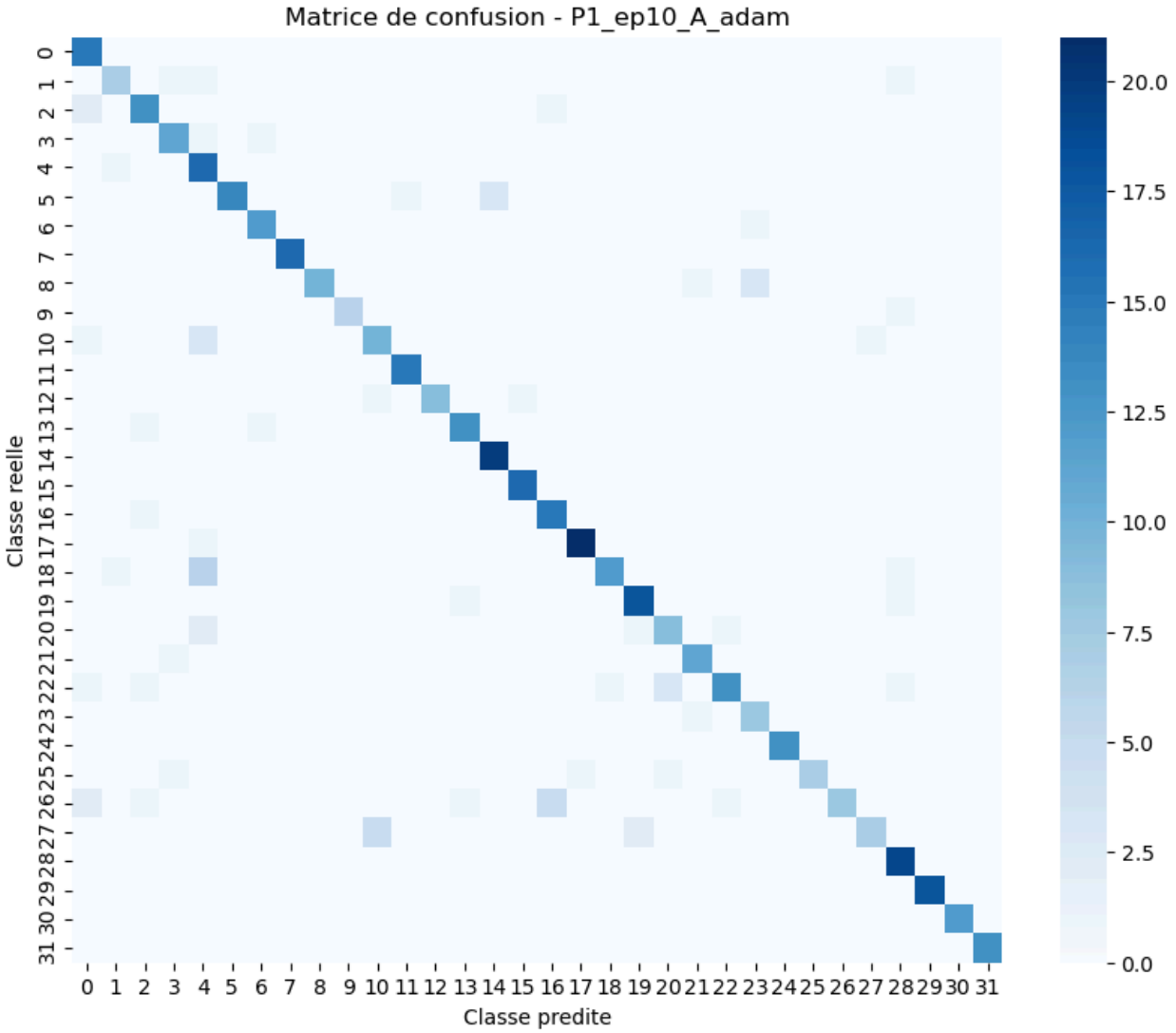
    y_true = []
    y_pred = []

    for images, labels in best_test_ds:
        preds = best_model.predict(images, verbose=0)
        y_true.extend(labels.numpy().tolist())
        y_pred.extend(np.argmax(preds, axis=1).tolist())

    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, annot=False, cmap='Blues')
    plt.title(f'Matrice de confusion - {best_exp_name}')
    plt.xlabel('Classe predite')
    plt.ylabel('Classe reelle')
    plt.show()

    print('Meilleure experience:', best_exp_name)
    print(classification_report(y_true, y_pred, target_names=best_class_names,
```

```
Found 4955 files belonging to 32 classes.
Using 3964 files for training.
Found 4955 files belonging to 32 classes.
Using 991 files for validation.
```


Matrice de confusion - P1_ep10_A_adam
Classe reelle
Classe predite
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0

Meilleure experience: P1_ep10_A_adam

	precision	recall	f1-score	support
ashanti blood	0.7143	1.0000	0.8333	15
barbados cherry	0.7778	0.7000	0.7368	10
beaumier du perou	0.7647	0.8125	0.7879	16
betel	0.7857	0.8462	0.8148	13
bitter orange	0.5333	0.9412	0.6809	17
caricature plant	1.0000	0.7778	0.8750	18
chinese guava	0.8571	0.9231	0.8889	13
chocolate tree	1.0000	1.0000	1.0000	16
chrysanthemum	1.0000	0.7143	0.8333	14
coeur demoiselle	1.0000	0.8571	0.9231	7
coffee	0.6250	0.6667	0.6452	15
croton	0.9375	1.0000	0.9677	15
duranta gold	1.0000	0.8182	0.9000	11
eggplant	0.8667	0.8667	0.8667	15
ficus	0.8696	1.0000	0.9302	20
fruitcitere	0.9412	1.0000	0.9697	16
geranium	0.7143	0.9375	0.8108	16
guava	0.9545	0.9545	0.9545	22
hibiscus	0.9231	0.6000	0.7273	20
jackfruit	0.8571	0.9000	0.8780	20
ketembilla	0.6923	0.6923	0.6923	13
lychee	0.8462	0.9167	0.8800	12
mulberry leaf	0.8667	0.6500	0.7429	20
papaya	0.6667	0.8889	0.7619	9
pimento	1.0000	1.0000	1.0000	13
pomme jacquot	1.0000	0.7000	0.8235	10
rose	1.0000	0.4444	0.6154	18
star apple	0.8750	0.5000	0.6364	14
sweet olive	0.7917	1.0000	0.8837	19
sweet potato	1.0000	1.0000	1.0000	18
thevetia	1.0000	1.0000	1.0000	12
vieux garcon	1.0000	1.0000	1.0000	13
accuracy			0.8479	480
macro avg	0.8706	0.8471	0.8456	480
weighted avg	0.8705	0.8479	0.8451	480